



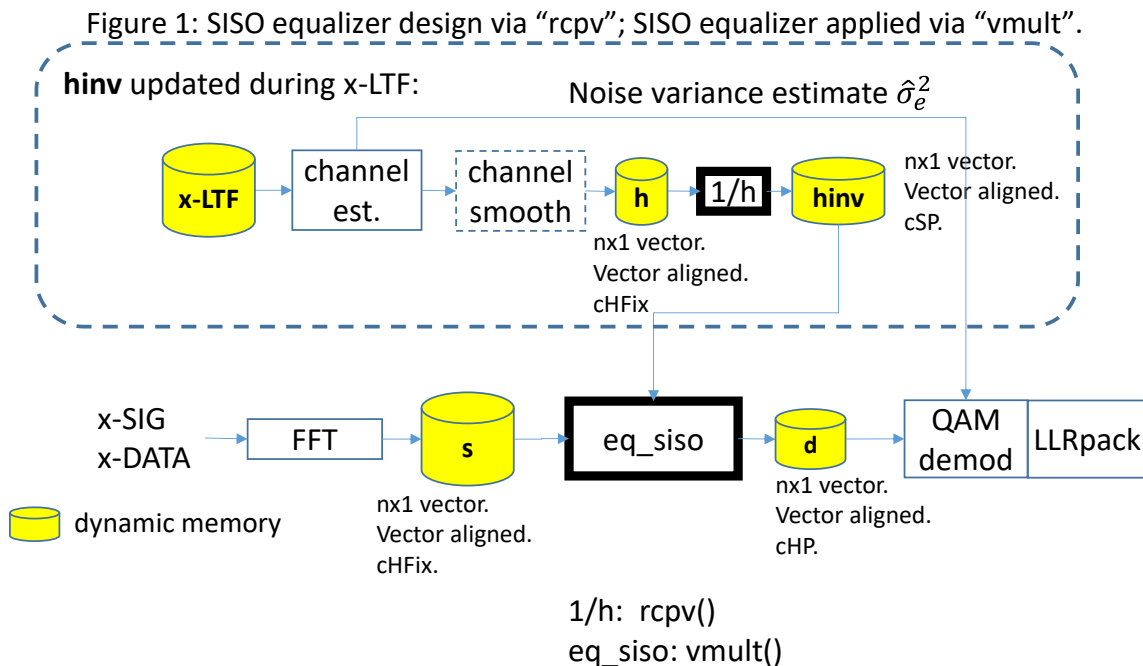
Name of the module: vector
 Target delivery date:
 Location in GIT repository: vspa_common\vspa-lib\vector

Overview:

This module uses two general linear algebra functions in “vector” library to accomplish equalizer weights design from received L-LTF and equalization for SIG & Data symbols in SISO mode:

1. “rcpv” function finds equalizer weights by $\mathbf{hinv} = \text{conj}(\mathbf{h}) ./ \text{abs}(\mathbf{h}).^2$, where the vector $\mathbf{h}$ is the CSI estimated from LTF symbols.
2. “vmult” equalizes subcarriers of each DATA or SIG symbol in an OFDM OFDM WiFi frame based on channel state information (CSI) estimated from LTF symbols: $\mathbf{d} = \mathbf{s} .* \mathbf{hinv}$, where the vector $\mathbf{d}$ represents the equalized subcarriers, $\mathbf{s}$ represents subcarriers produced by FFT of a DATA or SIG symbol, $\mathbf{hinv}$ is the output of rcpv() function.

The position of this module in RX system is shown below:



Function API:

“vmult” family:

```
extern void chp_chf_csp_vmult_asm( vspa2_complex_float16* py,
vspa2_complex_fixed16* px1, vspa_complex_float32* px2, size_t L );
```



```
#pragma cplusplus on
static inline void vmult(vspa2_complex_float16* py, vspa2_complex_fixed16*
px1, vspa_complex_float32* px2, size_t L) { chp_chf_csp_vmult_asm(py, px1,
px2, L); }
// ... more functions can be added with different in/out data types.
#pragma cplusplus off

"rcpv" family:
extern void csp_chf_rcpv_asm(vspa_complex_float32* py, vspa2_complex_fixed16*
px, size_t L);

#pragma cplusplus on
static inline void rcpv(vspa_complex_float32* py, vspa2_complex_fixed16* px,
size_t L) { csp_chf_rcpv_asm(py, px, L); }
static inline void rcpv(vspa_complex_float32* py, vspa_complex_float32* px,
size_t L) { csp_csp_rcpv_asm(py, px, L); }
// ... more functions can be added with different in/out data types.
#pragma cplusplus off
```

API description:

"rcpv" contains a family of elementwise vector-reciprocal functions for different input/output data types. In SISO equalizer application, the "chp_chf_csp_vmult_asm" method of this function is called after channel estimation.

"vmult" contains a family of element-wise vector multiplication functions for different input/output data types. In SISO equalizer application, the "csp_chf_rcpv_asm" method of this function is called after FFT of SIG or DATA.

Table 1: Function inputs of vmult.

px1	Pointer to x1, the FFT output of an OFDM symbol. Vector-aligned. In complex 16-bit half-fixed.
px2	Pointer to x2, the "1/h", which is the output of "rcpv". Vector-aligned. In complex 32-bit single-precision floating-pt.
L	The number of input DMEM lines. L >= 1.

Table 2: Function outputs of vmult.

py	Pointer to y, the equalized output. Vector-aligned. In complex 16-bit half-floating.
----	--

Table 3: Function inputs of rcpv.

px	Pointer to x = h, the channel estimate. Vector-aligned. In complex 16-bit half-fixed.
L	The number of input DMEM lines. L >= 1.

Table 4: Function outputs of rcpv.

py	Pointer to y, the "1/h". Vector-aligned. In complex 32-bit single-precision floating-pt.
----	--



DMEM requirements:

Table 5: DMEM requirements of vmult.

"vmult" Variable	Minimum size (words)	Minimum alignment (words)	Allocated by
y	32	32 (vector-aligned on 16AU)	Caller
x1	32	32	Caller
x2	32	32	Caller
L	1	1	Caller

Table 6: DMEM requirements of rcpv.

"rcpv" Variable	Minimum size (words)	Minimum alignment (words)	Allocated by
y	32	32	Caller
x	32	32	Caller
L	1	1	Caller

Implementation:

Theoretically, one line of CHF input requires {1 load, 2 stores} and {1 cmad, 1 rcp, 1 rmad}, hence, 3 cycle per input line. The following options are considered for "csp_chf_rcpv_asm" to try to reach the 3 cycle per input DMEM line. Option 4 is accepted, reaching 4 cycle per input DMEM line:

Table 7: Implementation options of rcpv.

	options	decision
1	Let "rmad.sau" take "rcp" output, thus avoiding S1 conflict when scheduling "rcp" and "xmad" in the same cycle.	Reject. This doesn't work because rcp only generates 32x1 real SP output, but "rmad" requires duplicating 32x1 into 64x1 to produce 64x1 cSP in one cycle. The use of "rmad.sau" does not provide opportunity of using S1r2c mode for duplication.
2	Contiguously loading 2, 4, or 8 lines to make pipeline tight.	Reject. There are not enough VRA registers to hold intermediate results of $ x .^{.2}$ and $1./ x .^{.2}$ while still keeping "x" for the final "rmad".
3	3-stage method: (1) $ x .^{.2}$ (2) $1./ x .^{.2}$ (3) $\text{conj}(x) .* 1./ x .^{.2}$.	Reject.



	Each stage uses one loop to process any L.	Although this method is easiest to debug and maintain but each stage introduces 12-cycle pipeline delay.
4	Merge 3 operations into one long pipeline with VRA registers to keep intermediate results because any store incurs 4-cycle overhead if in single-prec and 2-cycle overhead if in half-prec. One extra “load” has to be used in the main loop because there are two Smode switch and prec switch in the loop. Even if VRA registers are enough, 4 opB instructions cannot fit into 3-line loop. Note in the loop, opA space is fully occupied by one load and two stores. This method supports any $L \geq 1$.	Accept.
5	Hard coded to support only $L = \{2, 4, 8\}$.	Reject. This uses less cycles than option 4, but it is hard-coded. When L is big, we will run into VRA register space issue. As “rcpv” is only called once per frame, MCPS will not be high. It is better to design a general “rcpv” function.

Implementation of “chp_chf_csp_vmult_asm”:

Theoretically, one line of CHF input requires {3 loads, 1 store} and {2 cmad}, hence, 4 cycle per input line. Although “hinv” (in cSP) does not change, this function is called one symbol at a time. There is no possibility of holding “hinv” in VRA to avoid 2 loads.

The implementation targets 4 cycle-loop and supports any $L \geq 1$.

The operation is DMEM-access heavy.

Test plan:

The following table lists possible number of lines when “rcpv” and “vmult” are used for SISO equalization in 802.11a/n/ac/ax.

Table 8: “rcpv” and “vmult” tests via SISO equalization in 802.11a/n/ac/ax.

ch BW	20 MHz	40 MHz	80 MHz
oversampling factor	1	1	1



	Nfft	64	128	256
L-DATA	Nsd (# data SC)	48		
	Nsp (# pilot)	4		
	nLine cSP exact	3		
	nLine cHP exact	2		
HT-DATA	Nsd (# data SC)	52	108	
	Nsp (# pilot)	4	6	
	nLine cSP exact	4	6.75	
	nLine cHP exact	2	4	
VHT-DATA	Nsd (# data SC)	52	108	234
	Nsp (# pilot)	4	6	8
	nLine cSP exact	4	7	15
	nLine cHP exact	2	4	8
HE-DATA	Nsd (# data SC)	234	468	980
	Nsp (# pilot)	8	16	16
	nLine cSP exact	15	30	62
	nLine cHP exact	8	15	31
Summary				
cHalf	11a,n,ac	nL = {2,4,8} lines		
	11ax	nL = {8,15,31} lines		
cSP	11a,n,ac	nL = {3,4,7,15} lines		
	11ax	nL = {15,30,62} lines		

Currently we only consider tests on subcarriers stored in half-precision. L test set is {1,2,3,4,8,15,31}.

References:

NXP "WiFi PHY subsystem" white paper

ChEst_Implementation plan

Table 20-8, 802.11-2012

Table 22-8, 802.11ac-2013

Table 26-14, 802.11ax D0.5